Runtime as Living Documentation

Leverage the runtime system itself—such as service registries, Kubernetes labels, or logging/monitoring metadata—as the single source of truth for documentation and context.

Instead of maintaining separate wikis, diagrams, or spreadsheets (which inevitably drift out of sync), you embed meaningful, queryable metadata directly into the runtime using a thoughtfully designed taxonomy of tags/labels (e.g., domain=quotation, layer=backend, audience=regulator).

This transforms your infrastructure into a self-documenting system: teams query the live environment to discover services, dependencies, ownership, or even business capabilities, all while using the same tools and processes they already rely on (e.g., kubectl, monitoring dashboards).

The benefits include:

- accuracy (documentation can't diverge from reality, even after refactoring or architectural changes)
- integration (no context-switching to external tools)
- consistency (tags evolve with the system, enabling real-time impact analysis or debugging).

This requires crafting a taxonomy that aligns with the most typical queries, so that developers, ops, and even product teams can ask questions like "Which v2 services in the 'risk-scoring' domain are used by regulators?" directly from the runtime. The system becomes not just a deployment target, but a queryable knowledge base in itself.

Example Using Kubernetes

Discovery services (like Consul or Kubernetes) offer built-in queries for common needs, such as:

- List All Resources (kubectl get pods --all-namespaces)
- Check Pod Status (kubectl get pods -o wide)
- Filter by Creation Timestamp (kubectl get pods --field-selector=metadata.creationTimestamp>=2026-01-20T00:00:00Z)
- Get Service Endpoints (kubectl get endpoints <service-name>)
- Check Resource Usage (kubectl top pods --all-namespaces --sort-by=cpu)
- Get All Services by Specific Label (kubectl get pods -l <key=value>)

Some labels are quite commonly used, like version: and environments:

```
| env        | production, staging, development, testing, sandbox |
| version    | v1, v2, v3, legacy, canary                         |
```

You can use them to query the runtime system conveniently:

```
kubectl get pods -l env=staging --all-namespaces
kubectl get pods -l version=v2 --all-namespaces
```

The idea is to extend the system with custom tags (key=value labels here) to make it more knowledge-complete, and as a result, more powerful to help you answer your questions.

Group Exercise

Spoiler alert: don't read the next sub-section before finishing this exercise

Here's an overview of a given software system:

```
order-service: 5x, CRITICAL, postgres
user-auth: 3x, HIGH, sec-team
payment-gw: 4x, CRITICAL, rolling-deploy
notifications: 2x, MEDIUM, kafka
risk-engine: 3x, regulator, v3
quotes: 6x, backend
search: 2x, HA, risk-managers
reporting: 2x, experimental
billing: 3x, CRITICAL
CORE: v23, backend-team, legacy, claim management
api-gateway: 5x, CRITICAL
analytics: internal, data, snowflake
ext-api-proxy: 3x, MEDIUM, external
web-frontend: 8x, b2c
customer-area-bff: redis-cache, node
monitoring: platform-team, blue-green-deploy
```

Your task: inspired by this system:

- **Imagine 5-10 real-life situations** where you need to find where to investigate something: eg "list all services that are for regulators"
- From these use-cases, suggest as many **taxonomies** as needed that would make it easy and instant to get the answers you need: eg introduce a label "audience=regulator" in the discovery.
- Propose some (3-4) **sample queries** (using Kubernetes syntax or any other similar syntax) to illustrate the approach and its value.
- (Bonus) Discuss **where to source the labels** in the first place, and how to have it end up into the discovery registry.

Designing Your Own Custom Taxonomies

Key	Example Values
domain	`quotation`, `risk-scoring`, `insurance`, `claim-management`, `billing`
layer	`frontend`, `backend`, `data`, `platform`, `api-gateway`
audience	`b2b`, `b2c`, `c2c`, `regulator`, `internal`
sla	`critical`, `high`, `medium`, `low`, `best-effort`
team	`backend-team`, `frontend-team`, `data-team`, `platform-team`, `security`
deployment	`rolling-deploy`, `blue-green-deploy`, `legacy`
depends-on	`postgres`, `redis`, `kafka`, `external-api`, `legacy-system`
update-status	`pending`, `rolling-out`, `stable`, `deprecated`, `experimental`

Using these taxonomies and other built-in queries you can mix the query criteria for your needs:

- Find All Backend Services for "Quotation" in Production for B2B (Debugging a B2B quotation issue in production.)

```
kubectl get pods
-l layer=backend,domain=quotation,env=production,audience=b2b
--all-namespaces
```

- List Critical Data Services for Regulators Using PostgreSQL (Ensuring compliance for regulator-facing data services)

```
kubectl get pods
-l layer=data,sla=critical,audience=regulator,dependendency=postgres
--all-namespaces
```

- Find All V2 Frontend Services in Staging with Pending Updates (Preparing for a canary release in staging)

```
kubectl get pods
-l layer=frontend,version=v2,env=staging,update-status=pending
--all-namespaces
```

Sourcing the Knowledge to the Runtime

To leverage runtime systems as living documentation we must embed meaningful, queryable metadata directly into the infrastructure—using tools like Kubernetes or Consul. But these metadata must come from somewhere, which can be the source code, or some other tool.

For example:

- **Backstage** (Software Catalog) serves as the source of truth for team ownership

- Each module **README file** defines SLAs and operational expectations, ensuring clarity for developers and operators.
- **Code and annotations** (e.g., `@Domain("claim-management")`, `@Criticality("HIGH")`) are the default for all intrinsic metadata, ie. anything tied to the module's logic or requiring developer action. This ensures metadata evolves alongside the code and is version-controlled.
- **Infrastructure-as-Code** (IaC) tools like Terraform, Helm, or Kubernetes manifests are the source the truth for everything deployment.

In many cases, the metadata (labels) must be propagated from one place to another; this can be done using several mechanisms such as:

- Helm templating dynamically injects labels (e.g., from annotations or `values.yaml`) into Kubernetes manifests during deployment, bridging code and runtime.
- Hardcoded label registration on startup (e.g., services self-reporting to Consul) ensures real-time accuracy for dynamic attributes.
- Prometheus relabeling handle runtime-dependent labels (e.g. `environment=canary`), reducing manual overhead.

This approach balances developer autonomy (for module-specific metadata) with centralized governance (for cross-cutting concerns), while using automation (CI/CD, Helm, Prometheus) to keep everything in sync. The result: a **queryable, accurate, and maintainable** system where documentation lives alongside the code and runtime.